1. Predice el valor devuelto por las siguientes sentencias antes de ejecutarlas en la consola de las DevTools (F12):

Ejercicio 1 - sentencias a predecir

```
1. console.log(0 || "Fallback");
2. console.log(0 ?? "Fallback");
3. console.log(5 + "5");
4. let i = 2; console.log(i++ * ++i);
```

1. `console.log(0 || "Fallback");` devolverá `Fallback`.

   Explicación: El operador `||` devuelve el primer operando truthy. Como `0` es un valor falsy en JavaScript, se evalúa el segundo operando.

2. `console.log(0 ?? "Fallback");` devolverá `0`.

   Explicación: El operador `??` solo devuelve el operando derecho si el izquierdo es `null` o `undefined`. Como `0` no es ninguno de esos, se devuelve `0`.

3. `console.log(5 + "5");` devolverá `"55"` (un string).

   Explicación: El operador `+` con un número y un string realiza concatenación, no suma aritmética. JavaScript convierte el `5` numérico a string y lo une con `"5"`.

4. `let i = 2; console.log(i++ * ++i);` devolverá `8`

   Explicación paso a paso:

   - `i = 2`
   - `i++` → devuelve el valor actual (`2`) y luego incrementa `i` a `3`.
   - `++i` → incrementa `i` de `3` a `4` y devuelve `4`.
   - Multiplicación: `2 * 4 = 8`

---

2. Ejercicio 2. Verificación exhaustiva de variables y operadores

- Crea un script llamado `verificacion_tipos.js` y completa el bloque de código para analizar en la consola del navegador los comportamientos de asignación, comprobación y conversión de tipos:

verificacion_tipos.js

```
// 1. Demostración de ámbito de bloque e inmutabilidad
var variableGlobal = "Accesible fuera";
{
  let variableLocal = "Aislada en bloque";
  const CONFIG_APP = { version: "1.0.0", entorno: "test" };

  // Mutabilidad de propiedad en objeto declarado con const
  CONFIG_APP.entorno = "produccion";
  console.log("Objeto modificado:", CONFIG_APP);
}

console.log("variableGlobal fuera del bloque:", variableGlobal);
try {
  console.log("variableLocal fuera del bloque:", variableLocal);
} catch (e) {
  console.log("variableLocal fuera del bloque:", e.name + ": " + e.message);
}

// 2. Comprobaciones con el operador typeof
console.log("typeof 50n:", typeof 50n);        // bigint
console.log("typeof Symbol():", typeof Symbol()); // symbol
console.log("typeof null:", typeof null);     // object (anomalía)
console.log("typeof NaN:", typeof NaN);         // number

// 3. Coerción y comparaciones
console.log("5 == '5':", 5 == "5");          // true (igualdad abstracta)
console.log("5 === '5':", 5 === "5");         // false (igualdad estricta)
console.log("5 + '5':", 5 + "5");           // "55" (concatenación)
console.log("5 - '5':", 5 - "5");          // 0 (conversión numérica)

// 4. Operadores especiales in, delete e instanceof
const alumno = { id: 1001, curso: "2 DAW", activo: true };
console.log("¿Tiene propiedad 'curso'?:", "curso" in alumno); // true
delete alumno.activo;
console.log("Objeto tras borrado:", alumno);
console.log("alumno es instancia de Object:", alumno instanceof Object); // true
```

Los huecos que faltaban en cada `console.log` eran estas expresiones:

Sección	Expresión añadida	Resultado
1. Ámbito e inmutabilidad	`CONFIG_APP`	`{ version: "1.0.0", entorno: "produccion" }`
2. `typeof`	`typeof 50n`	`"bigint"`
2. `typeof`	`typeof Symbol()`	`"symbol"`
2. `typeof`	`typeof null`	`"object"`
2. `typeof`	`typeof NaN`	`"number"`
3. Coerción	`5 == "5"`	`true`
3. Coerción	`5 === "5"`	`false`

Sección	Expresión añadida	Resultado
3. Coerción	`5 + "5"`	`"55"`
3. Coerción	`5 - "5"`	`0`
4. Operadores	`"curso" in alumno`	`true`
4. Operadores	`alumno instanceof Object`	`true`

Dentro del bloque he añadido también dos comprobaciones para que el apartado de ámbito se vea de verdad: `variableGlobal` se puede leer desde fuera del bloque, pero al intentar leer `variableLocal` desde fuera salta un `ReferenceError`, que es justo lo que demuestra que `let` y `const` tienen ámbito de bloque y `var` no.

Tres resultados merecen un comentario:

- **`typeof null` devuelve `"object"`.** Es una anomalía histórica del lenguaje que se mantiene por compatibilidad: cambiarla ahora rompería muchísimo código antiguo que comprueba tipos de esa forma.
- **`typeof NaN` devuelve `"number"`.** `NaN` (*Not a Number*) no significa "no es un número", sino "el resultado de una operación numérica no es representable"; por eso su tipo sigue siendo `number`.
- **`CONFIG_APP.entorno` cambia sin error aunque el objeto se declare con `const`.** `const` impide reasignar la referencia, no congela el contenido: las propiedades del objeto siguen siendo modificables.

- Salida en la consola del navegador:

Consola del navegador (F12)

```
Objeto modificado: { version: "1.0.0", entorno: "produccion" }
variableGlobal fuera del bloque: Accesible fuera
variableLocal fuera del bloque: ReferenceError: variableLocal is not defined
typeof 50n: bigint
typeof Symbol(): symbol
typeof null: object
typeof NaN: number
5 == '5': true
5 === '5': false
5 + '5': 55
5 - '5': 0
¿Tiene propiedad 'curso'?: true
Objeto tras borrado: { id: 1001, curso: "2 DAW" }
alumno es instancia de Object: true
```

---

3. Ejercicio 3. Validador de tipos

- Código base entregado: `laboratorio_operadores.html`

laboratorio_operadores.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Laboratorio: Tipos y Operadores en JavaScript</title>
  <style>
    body { font-family: monospace; margin: 30px; background-color: #1e1e1e; color: #d4d4d4; }
    .panel { max-width: 650px; background: #252526; padding: 25px; border-radius: 8px;
      border: 1px solid #3c3c3c; }
    h2 { color: #4ec9b0; margin-top: 0; }
    .fila { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; color: #9cdcfe; }
    input[type="text"] { width: 95%; padding: 8px; background: #3c3c3c; border: 1px solid
      #555; color: #ce9178; font-size: 14px; }
    button { padding: 8px 16px; background: #0e639c; color: white; border: none; cursor:
      pointer; border-radius: 4px; }
    button:hover { background: #1177bb; }
    .salida { margin-top: 20px; padding: 15px; background: #181818; border-left: 4px solid
      #4ec9b0; font-size: 13px; line-height: 1.6; }
    .resaltado { color: #dcdcaa; font-weight: bold; }
    .aviso { margin-top: 12px; padding: 10px; background: #3a2a00; border-left: 4px solid
      #dcdcaa; color: #dcdcaa; font-weight: bold; }
  </style>
</head>
<body>

  <div class="panel">
    <h2>Laboratorio CE 2.b: Comprobador de Expresiones</h2>

    <div class="fila">
      <label for="campoValor1">Operando A (ej. 0, "0", false, null):</label>
      <input type="text" id="campoValor1" value="0">
    </div>
    <div class="fila">
      <label for="campoValor2">Operando B (ej. false, "", undefined, "0"):</label>
      <input type="text" id="campoValor2" value="false">
    </div>

    <button id="btnEvaluar">Evaluar Expresiones y Comparaciones</button>

    <div id="pantallaSalida" class="salida">Pulsa el botón para procesar...</div>
  </div>

  <script src="laboratorio_operadores.js"></script>
</body>
</html>
```

- `laboratorio_operadores.js` con las dos modificaciones solicitadas:

laboratorio_operadores.js (modificado)

```
function procesarEvaluacion() {
  const txtA = document.getElementById("campoValor1").value.trim();
  const txtB = document.getElementById("campoValor2").value.trim();
  const visor = document.getElementById("pantallaSalida");

  // Conversión segura de cadenas literales a primitivos reales usando JSON.parse
  let valA, valB;
  try {
    valA = JSON.parse(txtA);
  } catch (e) {
    valA = txtA; // Si no es JSON válido, lo trata como string puro
  }

  try {
    valB = JSON.parse(txtB);
  } catch (e) {
    valB = txtB;
  }

  // Evaluaciones de tipos y operaciones
  const tipoA = typeof valA;
  const tipoB = typeof valB;
  const igualdadAbstracta = (valA == valB);
  const igualdadEstricta = (valA === valB);
  const operacionOr = valA || valB;
  const operacionNullish = valA ?? valB;

  // Modificación 1: operador AND y doble negación booleana
  const operacionAnd = valA && valB;
  const negacionDoble = !!valA;

  // Modificación 2: protección contra excedentes de entero con BigInt
  let avisoBigInt = "";
  if (typeof valA === "number" && valA > Number.MAX_SAFE_INTEGER) {
    avisoBigInt = `<div class="aviso">Aviso: Precisión comprometida. Utiliza BigInt (sufijo n)
      para operar en este rango</div>`;
  }

  visor.innerHTML = `
    <strong>Operando A:</strong> <span class="resaltado">${String(valA)}</span> (typeof:
      ${tipoA})<br>
    <strong>Operando B:</strong> <span class="resaltado">${String(valB)}</span> (typeof:
      ${tipoB})<br>
    ------------------------------------------------<br>
    <strong>Igualdad abstracta (A == B):</strong> ${igualdadAbstracta}<br>
    <strong>Igualdad estricta (A === B):</strong> ${igualdadEstricta}<br>
    <strong>Cortocircuito OR (A || B):</strong> ${String(operacionOr)}<br>
    <strong>Coalescencia nula (A ?? B):</strong> ${String(operacionNullish)}<br>
    <strong>Cortocircuito AND (A && B):</strong> ${String(operacionAnd)}<br>
    <strong>Doble negación booleana (!!A):</strong> ${negacionDoble}
    ${avisoBigInt}
  `;
}

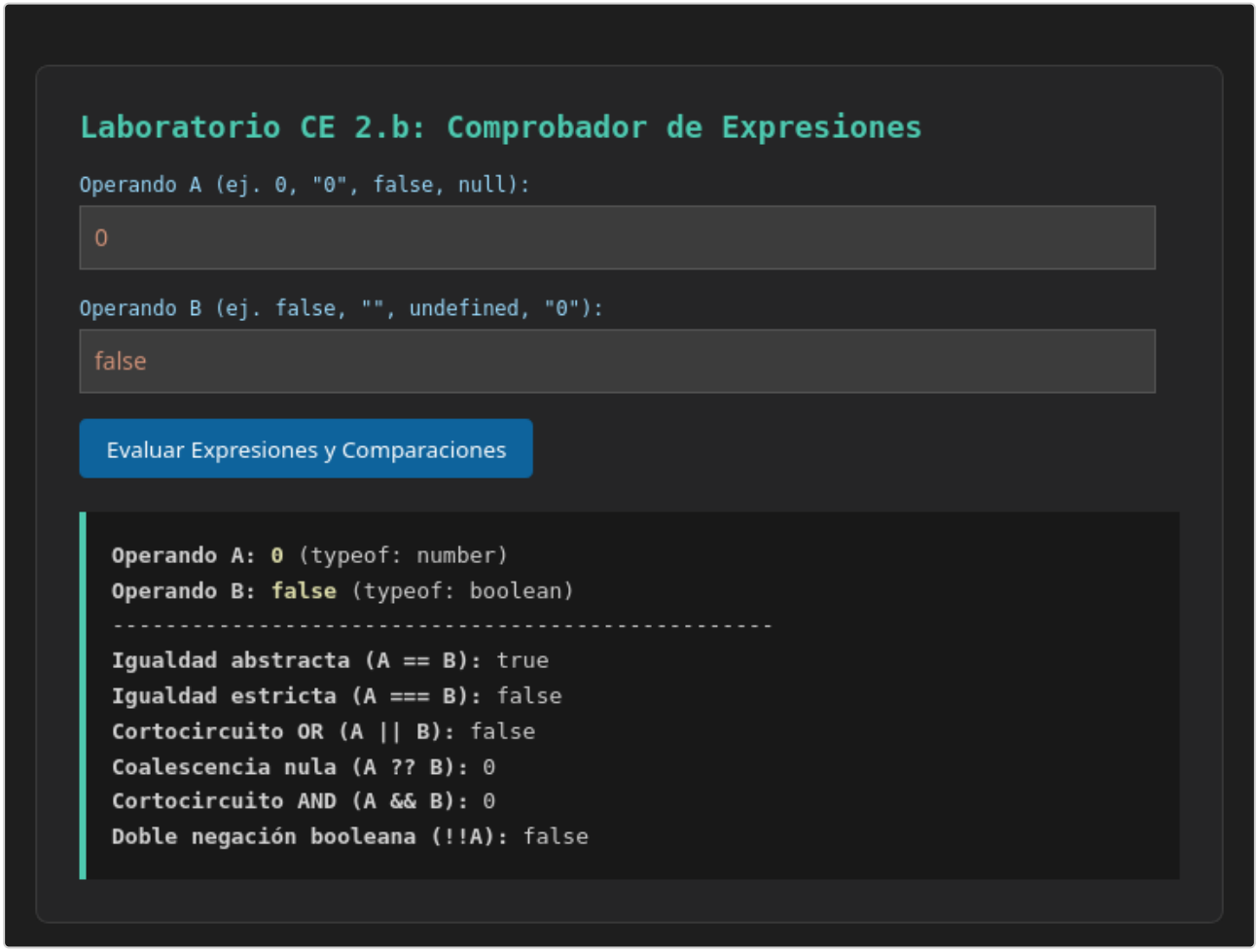
document.getElementById("btnEvaluar").addEventListener("click", procesarEvaluacion);
```

Modificación 1 (operador AND y negación booleana). Se añaden al cálculo las dos expresiones que faltaban y dos líneas nuevas en la salida:

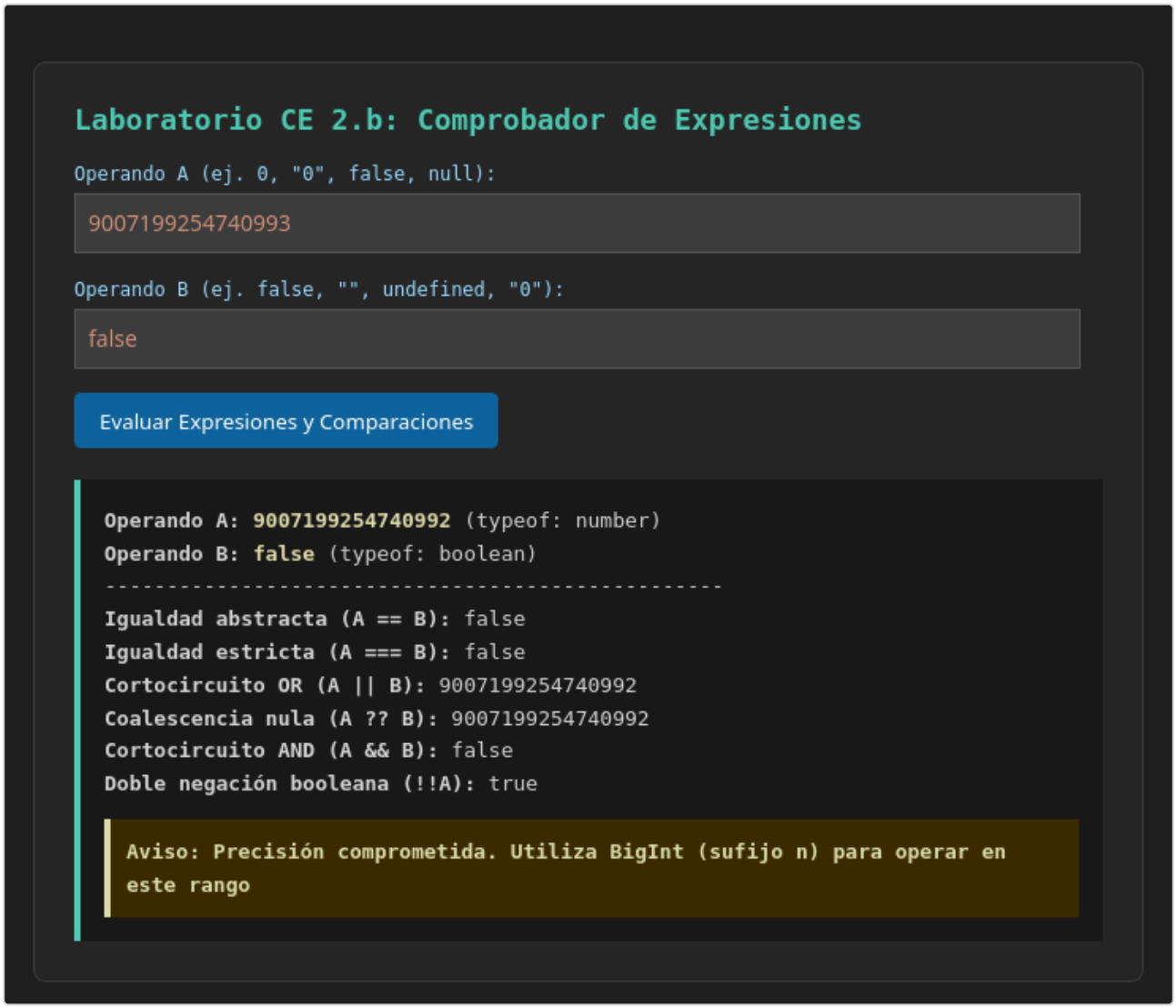
- `valA && valB`, que devuelve `valB` si `valA` es truthy y `valA` si no lo es.
- `!!valA`, que fuerza la conversión a booleano con la doble negación.

Modificación 2 (protección contra excedentes con BigInt). Antes de pintar la salida se comprueba `typeof valA === "number" && valA > Number.MAX_SAFE_INTEGER` y, si se cumple, se añade un bloque de aviso con el texto pedido.

- Funcionamiento con los valores por defecto (A = `0`, B = `false`):



- Modificación 2 en acción (A = `9007199254740993`):



La captura anterior es la mejor demostración de por qué hace falta la advertencia: se introduce `9007199254740993`, que está por encima de `Number.MAX_SAFE_INTEGER` (`9007199254740991`), y el propio laboratorio lo muestra como `9007199254740992`. El número ya se ha modificado solo al convertirlo a `number`, antes incluso de operar con él. Operando con `9007199254740993n` (sufijo `n`, tipo `bigint`) no se perdería precisión.

---

4. Ejercicio 4. Análisis de precedencia y operadores unarios combinados

- Calcula el valor final de `resultado`, `x` e `y` sin ejecutar el código:

Ejercicio 4 - secuencia a analizar

```
let x = 3;
let y = 4;
let resultado = x++ * ++y + (x & 1);
```

Resultado:

- `resultado = 15`
- `x = 4`
- `y = 5`

Paso a paso:

1. `x++` es post-incremento: devuelve el valor actual, `3`, y después sube `x` a `4`.
2. `++y` es pre-incremento: sube `y` de `4` a `5` y devuelve `5`.
3. La multiplicación se resuelve antes que la suma por precedencia: `3 * 5 = 15`.
4. `(x & 1)` se evalúa **después**, cuando `x` ya vale `4`. El operador `&` compara bit a bit; `4` en binario es `100` y `1` es `001`, así que el resultado es `0`.
5. Suma final: `15 + 0 = 15`.

Lo importante del ejercicio es el orden: los incrementos modifican `x` e `y`, pero el paréntesis `(x & 1)` no se lee hasta después, así que usa el valor ya incrementado de `x`.

---

5. Ejercicio 5. Comparativa de cortocircuitos con valores falsy críticos

- Indica qué imprimirá la función si un cliente VIP tiene asignado un descuento legítimo del 0% (`fijarDescuento(0)`):

Ejercicio 5 - funcion de descuento

```
function fijarDescuento(descuento) {
  let tasaFinalA = descuento || 15;
  let tasaFinalB = descuento ?? 15;
  console.log(`Tasa A: ${tasaFinalA} | Tasa B: ${tasaFinalB}`);
}
```

Salida:

```
Tasa A: 15 | Tasa B: 0
```

Razonamiento:

- `descuento || 15` con `descuento = 0` evalúa el operando izquierdo como falsy y salta al derecho, así que **`tasaFinalA` acaba valiendo `15`**. El `0%` del cliente se convierte en un `15%`, que es justo lo contrario de lo que se pretendía.
- `descuento ?? 15` solo recurre al valor de la derecha si el izquierdo es `null` o `undefined`. Como `0` no es ninguno de los dos, **`tasaFinalB` se queda en `0`**, respetando el descuento real.

La asignación correcta para el sistema es la segunda (`??`). Un descuento del `0%` es un dato válido y perfectamente intencionado, no una ausencia de dato, y `||` no sabe distinguir esas dos situaciones. El operador `??` se introdujo precisamente para cubrir este caso.

---

6. Ejercicio tipo test

1. ¿Qué valor almacena en memoria una variable declarada con `var` si todavía no se le ha asignado ningún dato (`var micoche;`)?

b) `undefined`.

Una variable declarada y no inicializada recibe automáticamente el valor `undefined`. El caso de `null` es distinto: hay que asignarlo a mano.

2. Si se evalúa la expresión aritmética `12 + "5"` en JavaScript, ¿qué resultado exacto se obtiene?

c) `"125"`.

El operador `+` con un operando de tipo string realiza concatenación, así que convierte el `12` a texto y lo une con `"5"`, dando el string `"125"`.

3. ¿Cuál de los siguientes operadores evalúa si dos operandos son estrictamente idénticos, exigiendo que coincidan tanto en valor como en tipo de datos sin permitir conversiones ocultas?

c) `===`.

El operador de igualdad estricta no realiza conversiones de tipo, a diferencia de `==`. Un solo `=` es asignación, y `eq` no es un operador de JavaScript.

4. ¿Qué valor devuelve la instrucción `typeof null` al ejecutarse en el motor del navegador?

c) `"object"`.

Devuelve `"object"` por un error de la primera implementación del lenguaje que se ha conservado por compatibilidad. No significa que `null` sea un objeto.

5. ¿Qué operador asigna un valor a una variable única y exclusivamente si dicha variable contiene `null` o `undefined`, respetando valores como `0` o `false`?

c) `??=`.

El operador de asignación con coalescencia nula solo asigna cuando el valor actual es `null` o `undefined`. Es la versión de asignación del operador `??` visto en el ejercicio 5.

6. Si una variable declarada con `const` contiene una matriz (`const notas = [5, 8];`), ¿cuál de las siguientes instrucciones es completamente legal y no provocará ningún error?

b) `notas.push(7);`

`const` impide reasignar la referencia, pero no impide modificar el contenido del array. Por eso `push` funciona, mientras que las opciones `a` y `d` (reasignar) y la `c` (redeclarar con `const`) lanzan un error.